Single-Source Shortest Paths in the Campus Shuttle Graph

Transit Analytics, Course Project

October 2026

1 Introduction

The campus shuttle network is modelled as a directed graph whose vertices are stops and whose edge weights are scheduled travel times in minutes. All weights are non-negative, so Dijkstra's algorithm applies directly. Algorithm 1 states the variant we implemented: it maintains tentative distances, repeatedly settles the unsettled vertex of minimum distance, and relaxes its outgoing edges until every reachable stop is settled.

2 Algorithm

The **while** l **do**
    o
end
op in Algorithm 1 runs until the queue is empty; the **if** t **then**
    e
end
st inside it discards stale entries left behind by earlier relaxations. Each edge is relaxed at most once per settling, which keeps the total work proportional to the number of queue operations.

3 Analysis

With a binary heap, initialisation costs $O(|V|)$, each extraction costs $O(\log |V|)$, and each relaxation costs $O(\log |V|)$ for the key decrease. The shuttle graph has 64 stops and 214 directed edges, so the whole computation finishes in well under a millisecond; the dominant cost in practice is loading the timetable, not the search.

Correctness follows the standard argument: when a vertex u is settled, $dist[u]$ already equals the true shortest-path distance, because any alternative path would have to leave the settled set through an edge of non-negative weight. The lazy-deletion **if** t **then**
    h
end
erefore never discards a vertex that could still improve the answer, and the returned array holds exact distances.

Algorithm 1: Dijkstra's algorithm with lazy deletion.

Input: Directed graph $G = (V, E)$ with non-negative weights w, source $s \in V$.
Output: Array *dist* with $dist[v]$ the shortest-path distance from s to v.

```
for each v ∈ V do
    dist[v] ← ∞;
    settled[v] ← false;
end
dist[s] ← 0;
Q ← priority queue of V keyed by dist;
while Q is not empty do
    u ← extract-minimum from Q;
    if settled[u] then
        skip u (a stale queue entry);
    end
    else
        settled[u] ← true;
        for each edge (u, v) ∈ E do
            if dist[u] + w(u, v) < dist[v] then
                dist[v] ← dist[u] + w(u, v);
                decrease the key of v in Q;
            end
        end
    end
end
return dist;
```